

Tutorial: String Concatenation in Java

Overview

String concatenation is the process of joining two or more strings together. In Java, you can concatenate strings using the `+` operator or the `StringBuilder`/`StringBuffer` classes for more complex or performance-sensitive operations.

1. Using the `+` Operator

The `+` operator is the simplest way to concatenate strings. When the `+` operator is used with strings, it joins them together.

Example 1: Simple Concatenation

```
public class Main {  
    public static void main(String[] args) {  
        String firstName = "John";  
        String lastName = "Doe";  
  
        String fullName = firstName + " " + lastName;  
        System.out.println(fullName); // Output: John Doe  
    }  
}
```

Example 2: Concatenating Strings with Other Data Types

```
public class Main {  
    public static void main(String[] args) {  
        String text = "The number is ";  
        int number = 10;  
  
        String result = text + number;  
        System.out.println(result); // Output: The number is 10  
    }  
}
```

2. Using the `concat()` Method

The `String` class provides a `concat()` method to concatenate strings. This method can only concatenate strings, not other data types.

Example 3: Using `concat()`

```
public class Main {  
    public static void main(String[] args) {  
        String str1 = "Hello, ";  
        String str2 = "world!";  
  
        String result = str1.concat(str2);  
        System.out.println(result); // Output: Hello, world!  
    }  
}
```

3. Using `StringBuilder` or `StringBuffer`

For more efficient string concatenation, especially in loops, use `StringBuilder` (non-thread-safe) or `StringBuffer` (thread-safe). These classes provide methods to append strings and other data types.

Example 4: Using `StringBuilder`

```
public class Main {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder();  
        sb.append("Hello, ");  
        sb.append("world!");  
        sb.append(" The number is ").append(10);  
  
        String result = sb.toString();  
        System.out.println(result); // Output: Hello, world! The number is  
10  
    }  
}
```

4. Concatenation in Expressions

Be mindful of the order of operations when concatenating strings with other data types.

Example 5: Order of Operations

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 10;  
        String text = "Result: ";  
  
        String result1 = text + a + b;  
        String result2 = text + (a + b);  
  
        System.out.println(result1); // Output: Result: 510  
        System.out.println(result2); // Output: Result: 15  
    }  
}
```

```
}  
}
```

5. Performance Considerations

- **Using + Operator:** Simple and convenient for small and simple concatenations.
- **Using `StringBuilder`:** More efficient for concatenating in loops or complex operations.
- **Using `StringBuffer`:** Use this if thread safety is required in concurrent environments.

Example 6: Performance with `StringBuilder`

```
public class Main {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder();  
  
        for (int i = 0; i < 100; i++) {  
            sb.append("Number ").append(i).append(" ");  
        }  
  
        String result = sb.toString();  
        System.out.println(result);  
    }  
}
```

Summary

- Use the `+` operator for simple concatenations.
- Use the `concat()` method for string-only concatenations.
- Use `StringBuilder` or `StringBuffer` for efficient concatenation in loops or complex operations.
- Pay attention to the order of operations when concatenating strings with other data types.